UNIFECAF — Engenharia da Computação
Model Based Design for Cyber-Physical Systems

Operação Linha Viva

Modelagem, controle, verificação formal e geração de código da junta do cotovelo do braço colaborativo AuroraArm

Relatório técnico — parte teórica, parte prática e memorial de cálculo (Anexo A)

Kaio Santana Rodrigues Santos — RA 98328
Repositório: https://github.com/kaiooo20/auroraarm-mbd

São Paulo, outubro de 2026

Sumário

1. Introdução e resumo dos resultados

PARTE TEÓRICA

2. Modelagem física da junta
3. Requisitos: desempenho x segurança colaborativa
4. Fundamentação das escolhas de controle, verificação e geração de código

PARTE PRÁTICA

5. Organização do repositório e reprodução
6. Modelo da planta, identificação e validação
7. Controlador: projeto, saturação e anti-windup
8. Supervisor de segurança e verificação formal
9. Testes baseados em modelo e cobertura
10. Geração de código C e equivalência SIL
11. Matriz de rastreabilidade
12. Honestidade técnica: o que o pacote sustenta e o que falta

Referências

ANEXO A — Memorial de cálculo, itens (a) a (e)

1. Introdução e resumo dos resultados

Um protótipo anterior de célula colaborativa da Aurora Robótica sofreu um quase-acidente: o torque da junta não foi zerado a tempo quando um operador aproximou a mão do efetuador, porque o firmware escrito à mão não tratava uma condição de borda entre os estados de movimento e de parada de segurança. Não havia modelo formal nem evidência reprodutível de que os requisitos de segurança fossem satisfeitos. Este trabalho entrega, para a junta do cotovelo do AuroraArm, o pacote de evidências exigido para a auditoria de segurança: **modelo** em espaço de estados identificado e validado, **controlador** com saturação e anti-windup, **supervisor de segurança** verificado exaustivamente por model checking, **código C gerado** a partir do modelo com equivalência numérica demonstrada em SIL e **matriz de rastreabilidade** de requisito a teste.

O método é o mesmo construído ao longo da disciplina para o NexaBot, aplicado a um novo sistema. Tudo é reprodutível com dois comandos (seção 5); cada número citado aqui é lido de `resultados/memorial.json`, gerado por `scripts/memorial.py`.

Requisito	Exigido	Obtido	Como foi demonstrado
REQ-CTRL-AA-001	erro em regime nulo	1.2e-04 rad	simulação, 9 cenários
REQ-CTRL-AA-002	sobressinal ≤ 8 %	2,81 % (pior caso)	simulação, 9 cenários + robustez
REQ-CTRL-AA-003	acomodação ≤ 300 ms	218 ms (pior caso)	simulação, 9 cenários + robustez
REQ-CTRL-AA-004	|V| ≤ 48 V, |i| ≤ 8 A	|i| ≤ 7,33 A; |V| ≤ 48 V	simulação + 5 000 entradas aleatórias
REQ-SAFE-AA-001	torque zero ≤ 100 ms	31,96 ms (pior caso)	model checking (32.011 estados) + varredura de 64.320 combinações
REQ-SAFE-AA-002	só religa com rearme	verdadeira	model checking (P2) + MBT
REQ-SAFE-AA-003	retorna à trajetória	verdadeira	model checking de vivacidade (P3)
REQ-SAFE-AA-004	v ≤ 250 mm/s na zona	212 mm/s em regime	model checking (P4) + simulação

O model checker encontrou na versão preliminar do supervisor (v0) um contraexemplo real de REQ-SAFE-AA-001 que reproduz o tipo de falha do protótipo anterior (seção 8,4); o código gerado é **bit a bit idêntico** ao modelo em precisão simples ao longo de 3.801 amostras (seção 10); o firmware ocupa 692 bytes (de 256 KB) e no máximo 3.91 µs por ciclo (de 200 µs),

PARTE TEÓRICA

2. Modelagem física da junta

A junta é um motor CC de ímã permanente de 48 V acoplado a uma transmissão harmônica 100:1. Todas as grandezas mecânicas são **referidas ao eixo do motor**: a inércia e o atrito da tabela já são os valores refletidos, e o torque de carga de 0,25 N m é o torque do payload (2 kg a 0,35 m) dividido pela redução. Ângulo e velocidade da junta são $\theta_j = \theta_m/N$ e $\omega_j = \omega/N$ (Siciliano et al., 2010, cap. 5).

2.1 Leis de conservação

Equação elétrica da armadura (lei das malhas de Kirchhoff — conservação de energia no circuito): a tensão aplicada se divide entre a queda resistiva, a queda indutiva e a força contraeletromotriz, proporcional à velocidade:

$$L \cdot di/dt = V - R \cdot i - K_e \cdot \omega$$

Equação mecânica do eixo (2ª lei de Newton na rotação — conservação do momento angular): o torque eletromagnético acelera a inércia contra o atrito viscoso e o torque de carga:

$$J \cdot d\omega/dt = K_t \cdot i - b \cdot \omega - T_L$$

O acoplamento eletromecânico aparece duas vezes, em $K_t \cdot i$ e em $K_e \cdot \omega$. Em unidades SI, $K_t = K_e$ (0,085 nos dados) é consequência da conservação de potência no entreferro: $K_t \cdot i \cdot \omega = K_e \cdot \omega \cdot i$.

2.2 Escolha das variáveis de estado

O estado de um sistema é o conjunto mínimo de variáveis que, junto com as entradas futuras, determina todo o comportamento futuro. Cada elemento armazenador de energia contribui com uma variável: a indutância armazena $\frac{1}{2} \cdot L \cdot i^2$ e a inércia armazena $\frac{1}{2} \cdot J \cdot \omega^2$. Por isso:

- $\mathbf{x_1 = i}$ (corrente): é a energia magnética e, por ser proporcional ao torque, é a grandeza limitada pelo driver (8 A) — precisa estar visível no modelo.
- $\mathbf{x_2 = \omega}$ (velocidade do eixo): é a energia cinética e aparece na f.c.e.m., que consome a maior parte da tensão no ponto crítico (38,25 de 41,24 V).
- Para o laço de posição acrescenta-se $\boldsymbol{\theta_m}$ ($d\theta_m/dt = \omega$), um integrador puro que não armazena energia mas é a grandeza controlada (modelo de 3 estados em `auroraarm/planta.py::matrizes_posicao`).

Ambas as variáveis são mensuráveis (sensor de corrente do driver e encoder), têm significado físico direto e tornam as matrizes lineares e invariantes no tempo. Resultado (memorial, item a):

$$\dot{x} = A \cdot x + B \cdot V + E \cdot T_L, \quad y = C \cdot x, \quad A = [-R/L \quad -K_e/L \; ; \; K_t/J \quad -b/J], \quad B = [1/L \; ; \; 0], \quad E = [0 \; ; \; -1/J], \quad C = [0 \quad 1]$$

Premissas do modelo: transmissão rígida (a flexibilidade da harmônica, relevante acima de algumas dezenas de Hz, não foi modelada — ver seção 12); atrito apenas viscoso (sem Coulomb); parâmetros constantes (sem aquecimento da armadura); torque de carga tratado como perturbação constante de ±0,25 N m (na prática varia com a configuração do braço); driver de ponte H ideal (tensão aplicada = comando, dentro de ±48 V).

2.3 Dinâmica resultante

Os autovalores de A são -644,4 e -22,62 rad/s, ambos reais e negativos: o motor em malha aberta é estável e sem oscilação. Eles definem duas constantes de tempo, $\tau_e = 1{,}552$ ms (elétrica) e $\tau_m = 44{,}2$ ms (mecânica), separadas por um fator 28,5. Essa separação de escalas de tempo justifica três decisões do projeto: (i) identificar as equações elétrica e mecânica em etapas diferentes (seção 6); (ii) projetar o controlador de posição sobre um modelo reduzido

de 2ª ordem que ignora τ_e (seção 7); (iii) tratar o corte do torque como quase instantâneo frente ao prazo de 100 ms ($\leq 5\tau_e$ = 7,76 ms, seção 8).

3. Requisitos: desempenho x segurança colaborativa

Os dois grupos de requisitos têm naturezas diferentes e, por isso, exigem técnicas de demonstração diferentes:

Aspecto	Desempenho (REQ-CTRL-AA-001..004)	Segurança colaborativa (REQ-SAFE-AA-001..004)
O que restringe	qualidade da resposta contínua (erro, sobressinal, tempo, limites físicos)	comportamento discreto do sistema na presença de pessoas (estados, eventos, prazos)
Domínio	sinais contínuos, equações diferenciais	eventos discretos e tempo (autômatos temporizados)
Origem	engenharia de processo (produtividade)	ISO/TS 15066 e ISO 10218 (risco a pessoas)
Consequência da falha	peça mal montada, perda de ciclo	lesão do operador
Evidência adequada	simulação em cenários + robustez; testes	verificação EXAUSTIVA de todos os comportamentos possíveis
Por que não basta simular	o espaço de cenários relevante é pequeno e contínuo	a falha está em combinações raras de eventos (a do protótipo era uma condição de borda)

Na lógica temporal (Baier e Katoen, 2008, cap. 3), uma **propriedade de segurança** afirma que "algo ruim nunca acontece": toda violação tem um prefixo finito que a demonstra (um traço que termina no instante do erro). Uma **propriedade de vivacidade** afirma que "algo bom eventualmente acontece": nenhum prefixo finito a viola, porque o bom ainda pode acontecer depois; a violação é um traço infinito (na prática, um ciclo) em que o bom nunca ocorre.

Requisito	Classificação	Formalização	Justificativa
REQ-SAFE-AA-001: mão a < 500 mm → torque zero em ≤ 100 ms	**Segurança** (com prazo — "vivacidade limitada")	$\mathrm{AG}(\text{mão} \rightarrow \mathrm{AF}_{\leq 100\mathrm{ms}}\ \text{torque_zero})$, verificada como invariante: "nunca a mão fica mais de 100 ms no campo com torque ≠ 0"	Embora tenha a forma "eventualmente", o prazo a torna de segurança: se for violada, existe um instante finito (100 ms após a entrada da mão) em que o erro já é visível.
REQ-SAFE-AA-002: depois de zerado, não religa sem rearme	**Segurança**	$\mathrm{AG}((\text{PARADA} \vee \text{AGUARDA}) \wedge \neg\text{rearme} \rightarrow \mathrm{AX}\ \neg\text{habilita})$	A coisa ruim (religar sem rearme) é um evento observável numa única transição.
REQ-SAFE-AA-003: mão removida e rearme → volta à trajetória	**Vivacidade**	$\mathrm{AG}(\text{rearme} \wedge \neg\text{mão} \rightarrow \mathrm{AF}(\text{retomou} \vee \text{mão}))$ ("leads-to")	Não há prazo: nenhum prefixo finito viola. A violação seria ficar para sempre sem retomar — um ciclo no grafo de estados.
REQ-SAFE-AA-004: v ≤ 250 mm/s na zona colaborativa	**Segurança** (invariante)	$\mathrm{AG}(\text{zona} \wedge \text{habilita} \rightarrow \mathrm{AF}_{\leq 6\mathrm{ms}}\ \text{lim_vel}) + \mathrm{G}(v \leq 250\ \mathrm{mm/s})$ após a desaceleração	Ultrapassar a velocidade é um evento instantâneo detectável num prefixo finito.

Note o par de cuidados na formalização de REQ-SAFE-AA-001 e 004: um requisito em linguagem natural esconde premissas (o que é "detectada"? a partir de quando conta o prazo?). A seção 8.4 mostra que a primeira formalização do monitor estava errada e produziu um contraexemplo espúrio — um exemplo concreto de que formalizar é parte do trabalho de engenharia, e não um detalhe.

4. Fundamentação das escolhas

4.1 Controle (Unidades 1 e 2)

PID discreto em vez de realimentação de estados. A realimentação de estados exigiria a corrente na lei de controle a 500 Hz, mas τ_e/T_s = 0,78: a dinâmica elétrica termina dentro de

um único período e não pode ser controlada pelo laço de posição (memorial, item c). Com a planta efetivamente de 2ª ordem (integrador + polo mecânico), um PID com ação integral dá erro nulo para degrau mesmo com a carga constante (tipo 1 em relação à perturbação), e é o que a equipe da Aurora já conhece — vantagem de manutenção. Os ganhos partiram de alocação de três polos reais no modelo reduzido e foram refinados por otimização sobre o laço não linear (`scripts/sintonia.py`).

Estrutura 2-GDL. A derivada é aplicada só à medição (evita o "chute derivativo" no degrau) e filtrada (T_f = 1 ms); o termo proporcional usa ponderação de referência $\beta = 0{,}55$, que remove o zero de malha fechada responsável pelo sobressinal sem perder rejeição de perturbação.

Saturação e anti-windup. No ponto crítico a folga de tensão é de apenas 6,76 V (14,1 %) (item b): qualquer transitório satura o atuador. Durante a saturação o integrador continua acumulando erro (windup) e, ao sair dela, produz grande sobressinal. Usou-se **retrocálculo** (back-calculation): o integrador recebe $K_{aw}\cdot(u_{sat} - u)$. A saturação tem duas camadas: uma janela de tensão que mantém $|i| \le 7{,}5$ A, calculada a partir da f.c.e.m. estimada ($K_e\cdot\hat{\omega} \pm R\cdot i_{lim}$), e por último o limite rígido de ±48 V.

Discretização. A planta foi discretizada por segurador de ordem zero exato (exponencial de matriz) — é assim que o conversor PWM aplica a tensão; a simulação usa 20 subpassos por período só para registrar picos de corrente. O controlador é escrito diretamente em diferenças (Euler para o integrador, filtro de 1ª ordem para a derivada), forma que vai literalmente para o C.

4.2 Verificação (Unidade 3)

O supervisor foi modelado como uma máquina de estados síncrona e composto com autômatos temporizados do ambiente (mão, zona, rearme), do scanner (atraso não determinístico em [0, 20] ms), do RTOS (período de 2 ms com possível perda de um ciclo) e do driver (corte do torque em 8 ms). Por serem autômatos *fechados* (guardas $\le$ e $\ge$ com constantes inteiras), a semântica de relógios digitais com passo de 1 ms preserva as propriedades de alcançabilidade do tempo denso (Henzinger, Manna e Pnueli, 1992), o que permite uma exploração explícita e exaustiva em busca em largura — a mesma ideia do UPPAAL. Propriedades com prazo viraram **monitores** (relógios observadores, Baier e Katoen, 2008, cap. 9) e a vivacidade virou busca de **ciclos** no subgrafo de estados com obrigação pendente.

Escreveu-se um model checker próprio (`auroraarm/verificacao.py`, ~300 linhas) em vez de usar UPPAAL por três motivos: (i) o modelo do supervisor verificado é *exatamente* a tabela que gera o código C — não há tradução manual para outra linguagem; (ii) o checker roda no mesmo pytest, a cada mudança; (iii) o checador é ele próprio testado com mutantes (seção 9). O custo é que uma ferramenta própria não é qualificada (seção 12).

Os testes do supervisor foram **gerados do modelo** (MBT): caminhos que cobrem cada transição, todos os pares estado × entrada e passeios aleatórios com invariantes — e executados contra a implementação Python *e* o C gerado.

4.3 Geração de código e certificação (Unidade 4)

O controlador é um **modelo executável**: uma lista ordenada de passos de fluxo de dados (`controlador.PASSOS`). O mesmo modelo é interpretado em Python e traduzido para C por `auroraarm/codegen.py`, que imprime uma operação por linha na mesma ordem, em float IEEE-754 binário32 (a FPU do Cortex-M4 é de precisão simples), sem laços, recursão ou alocação dinâmica (tempo de execução constante, requisito típico da IEC 61508-3 para software de segurança), e com `-ffp-contract=off` (sem FMA, que mudaria os arredondamentos). Cada linha do C carrega a etiqueta do requisito que implementa, e o cabeçalho tem um bloco de rastreabilidade com o hash do modelo e dos parâmetros; um teste falha se o C versionado divergir do que o gerador produz hoje. A equivalência foi verificada em

SIL: o C compilado roda via ctypes com as mesmas entradas do modelo e também fecha a malha com a planta simulada. A co-simulação por FMI (fonte 6) não foi usada: como planta e controlador estão no mesmo ambiente Python, o SIL por ctypes cumpre o mesmo papel sem a camada de empacotamento FMU; numa integração com outra ferramenta (p. ex. o simulador da célula) o C gerado poderia ser empacotado como FMU de co-simulação.

4.4 Normas e caso real

A **ISO/TS 15066** define os modos colaborativos; este projeto combina *parada monitorada de segurança* (REQ-SAFE-AA-001/002) com *monitoramento de velocidade e separação* (REQ-SAFE-AA-004), e a distância do campo de alerta foi dimensionada com a velocidade de aproximação de 1,6 m/s da ISO 13855. A **IEC 61508** enquadra o supervisor como função de segurança e motiva as escolhas de código (subconjunto restrito, determinismo, rastreabilidade) e a discussão de qualificação de ferramentas e independência da seção 12.

Como caso real, adotou-se Colim et al. (2021), que relatam a reformulação de um posto de montagem manual em posto colaborativo com um **UR10e** (Universal Robots): avaliação de risco inicial, escolha do cobot pela reach, 6 GDL e sensor de torque embutido, garra a vácuo colaborativa, proteção física ISO 13857 apenas onde havia perigo térmico, e medição ergonômica antes/depois. Duas lições foram trazidas para cá: (i) a segurança é demonstrada por *avaliação de risco sistemática*, não pela escolha do robô — o cobot certificado não dispensa a análise da aplicação; (ii) o perigo dominante pode estar fora do braço (lá, a cola quente; aqui, a latência do scanner e do RTOS, que juntos respondem por 75 % do pior caso de REQ-SAFE-AA-001).

PARTE PRÁTICA

5. Organização do repositório e reprodução

Repositório: https://github.com/kaiooo20/auroraarm-mbd. Todos os caminhos citados neste relatório são relativos à raiz do repositório.

```
parametros.json            fonte única dos dados do enunciado
auroraarm/                 o MODELO (biblioteca Python)
  planta.py                espaço de estados, ZOH, simulação          -> seções 2, 6, memorial (a)(b)(c)
  identificacao.py         ensaio emulado, mínimos quadrados, validação -> seção 6
  controlador.py           PID 2-GDL + saturação + anti-windup (PASSOS) -> seção 7
  supervisor.py            tabela de transições v0 (com defeito) e v1  -> seção 8
  verificacao.py           model checker: P1..P4, contraexemplos       -> seção 8, memorial (d)
  testes_modelo.py         geração de testes a partir do grafo         -> seção 9
  codegen.py               gerador de C a partir do modelo             -> seção 10
  sil.py                   SIL via ctypes, equivalência numérica       -> seção 10, memorial (e)
gen/auroraarm_junta.{h,c}  código C GERADO (não editar)
sil/bench.c                medição de tempo no host
scripts/                   sintonia.py, memorial.py, figuras.py, rastreabilidade.py
tests/                     42 testes pytest (requisitos, MBT, SIL, model checking, codegen)
resultados/                memorial.json/.txt, contraexemplo_v0.txt, figuras/
docs/                      passo-a-passo.md (guia de estudo), rastreabilidade.md
```

```
pip install -r requirements.txt          # numpy, scipy, matplotlib, pytest, reportlab
python -m pytest                         # 42 testes: requisitos, MBT, SIL, model checking
python scripts/memorial.py               # recalcula o memorial (itens a..e) e o model checking
python scripts/figuras.py && python scripts/rastreabilidade.py && python relatorio/gerar_pdf.py
```

6. Modelo da planta, identificação e validação

As matrizes (memorial, item a) estão em `auroraarm/planta.py::matrizes` e são conferidas por `tests/test_planta.py`. O posto 2 das matrizes de controlabilidade e de observabilidade (com y = ω) garante que a tensão consegue levar a junta a qualquer estado e que a corrente pode ser reconstruída só a partir da velocidade — o que justifica usar um observador caso o sensor de corrente falhe (degradação considerada na seção 12). A simulação em malha aberta com V = 41,24 V e T_L = 0,25 N m converge para exatamente 450 rad/s e 3,735 A, validando o balanço de regime do item (b).

Identificação. Sem acesso à bancada, o ensaio foi emulado: tensão PRBS (níveis de 5 ms) e degrau de carga sobre a planta nominal, com ruído de 0,02 A na corrente e 0,5 rad/s no encoder. Aproveitando a separação de escalas, a equação elétrica foi estimada a 10 kHz por mínimos quadrados, com inversão exata da ZOH; a mecânica, na forma integral em janelas de 10 ms, o que filtra o ruído do encoder. A validação usou um *segundo* conjunto de dados (outra semente, outra amplitude, outra carga) em simulação livre:

Parâmetro	Nominal	Estimado	Erro
R	0.8	0.8009	0,11 %
L	0.0012	0.0012	-0,01 %
Ke	0.085	0.08499	-0,01 %
J	0.00042	0.0004188	-0,28 %
b	0.00015	0.0001513	0,84 %
Ajuste NRMSE (validação)		ω: 98,7 %	i: 99,8 %

Defeito encontrado e corrigido: a primeira versão estimava tudo de uma vez a 10 kHz e errou o atrito em 119 %, porque o ruído do encoder entra no regressor (erro nas variáveis) e a dinâmica mecânica quase não muda de uma amostra para a outra. O teste `tests/test_identificacao.py` exige erro < 2 % em todos os parâmetros.

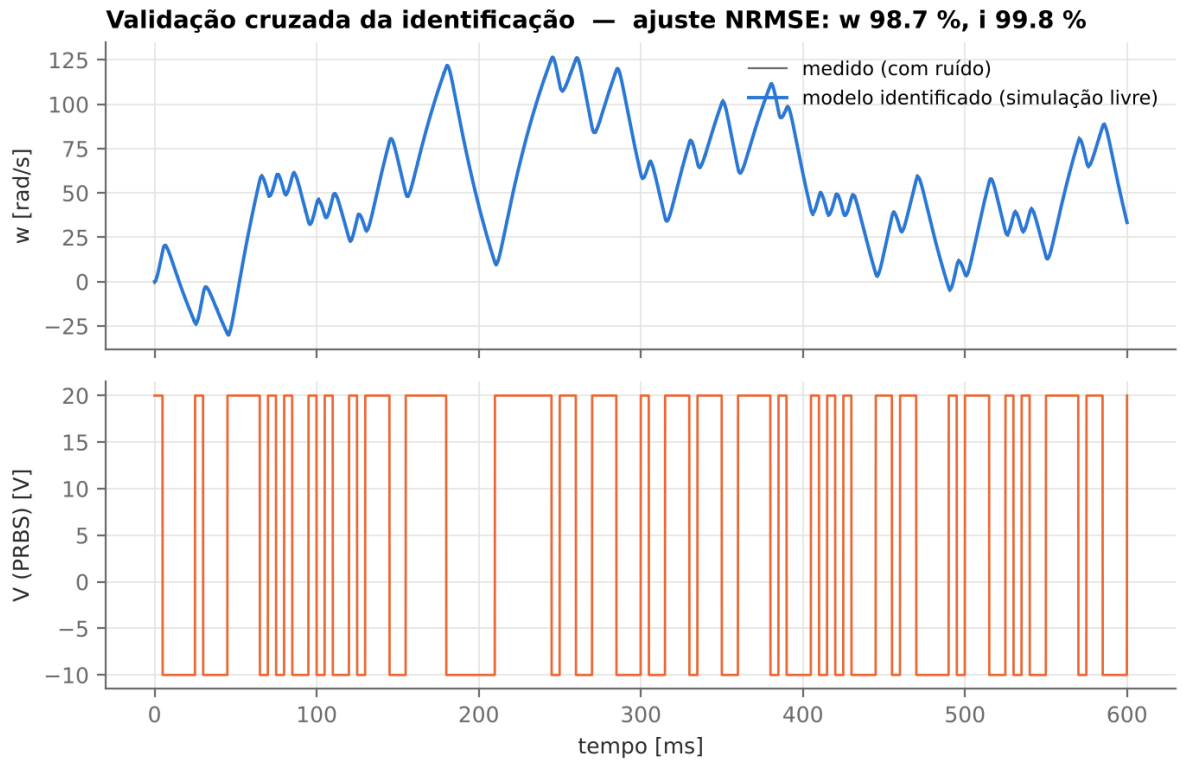


Figura 1 — Validação cruzada: o modelo identificado, simulado sem realimentar a medição, reproduz a velocidade medida num ensaio que não foi usado na estimação.

7. Controlador: projeto, saturação e anti-windup

Modelo reduzido para projeto (desprezando τ_e): $\theta_m(s)/V(s) = K/(s(\tau s+1))$, com $K = K_t/(Rb + K_tK_e) = 11{,}572$ rad/(V·s) e $\tau = RJ/(Rb + K_tK_e) = 45{,}75$ ms. Com PID, a equação característica é $\tau s^3 + (1 + K\,K_d)s^2 + K\,K_p\,s + K\,K_i = 0$; impondo três polos em −30 rad/s obtém-se o ponto de partida $K_p = 10{,}7$, $K_i = 107$, $K_d = 0{,}27$. O refinamento minimizou o pior tempo de acomodação sobre 9 cenários (degraus de ±10 e 2 rad no motor; carga de +0,25, 0 e −0,25 N m), penalizando sobressinal acima de 4 % (metade da folga do requisito). Ganhos finais, em `auroraarm/controlador.py::GANHOS_PADRAO`: $K_p = 10{,}1$ V/rad, $K_i = 125$ V/(rad·s), $K_d = 0{,}23$ V·s/rad, $T_f = 1$ ms, $\beta = 0{,}55$, $K_{aw} = 0{,}30$.

Premissa do ensaio de degrau: 0,1 rad na junta (≈ 5,7°, 10 rad no motor), aplicado com a junta já em equilíbrio sob carga. Esse degrau leva o laço ao limite de corrente; movimentos maiores do ciclo de pega-e-solta são feitos por trajetórias suaves, não por degraus. Fora desse envelope (degrau de 15 rad com carga a favor) o sobressinal chega a 15,4 % — limitação registrada.

Degrau (motor)	Carga [N m]	Sobressinal	t_s (2 %)	Erro regime	|V| máx	|i| máx
+10 rad	+0.25	0,54 %	218 ms	1.2e-04	11,9 V	7,33 A
+10 rad	+0.00	0,61 %	190 ms	5.8e-05	11,9 V	7,21 A
+10 rad	-0.25	1,88 %	162 ms	2.8e-05	11,4 V	7,32 A
-10 rad	+0.25	1,88 %	162 ms	2.8e-05	11,4 V	7,32 A
-10 rad	+0.00	0,61 %	190 ms	5.8e-05	11,9 V	7,21 A
-10 rad	-0.25	0,54 %	218 ms	1.2e-04	11,9 V	7,33 A
+2 rad	+0.25	0,86 %	150 ms	2.0e-06	6,9 V	7,32 A
+2 rad	+0.00	1,48 %	124 ms	1.1e-06	6,7 V	7,20 A
+2 rad	-0.25	2,81 %	188 ms	3.2e-06	6,6 V	7,04 A

Degrau (motor)	Carga [N m]	Sobressinal	t_s (2 %)	Erro regime	|V| máx	|i| máx
sem anti-windup	+0,25	94,3 %	não acomoda	—	22,1 V	7,33 A

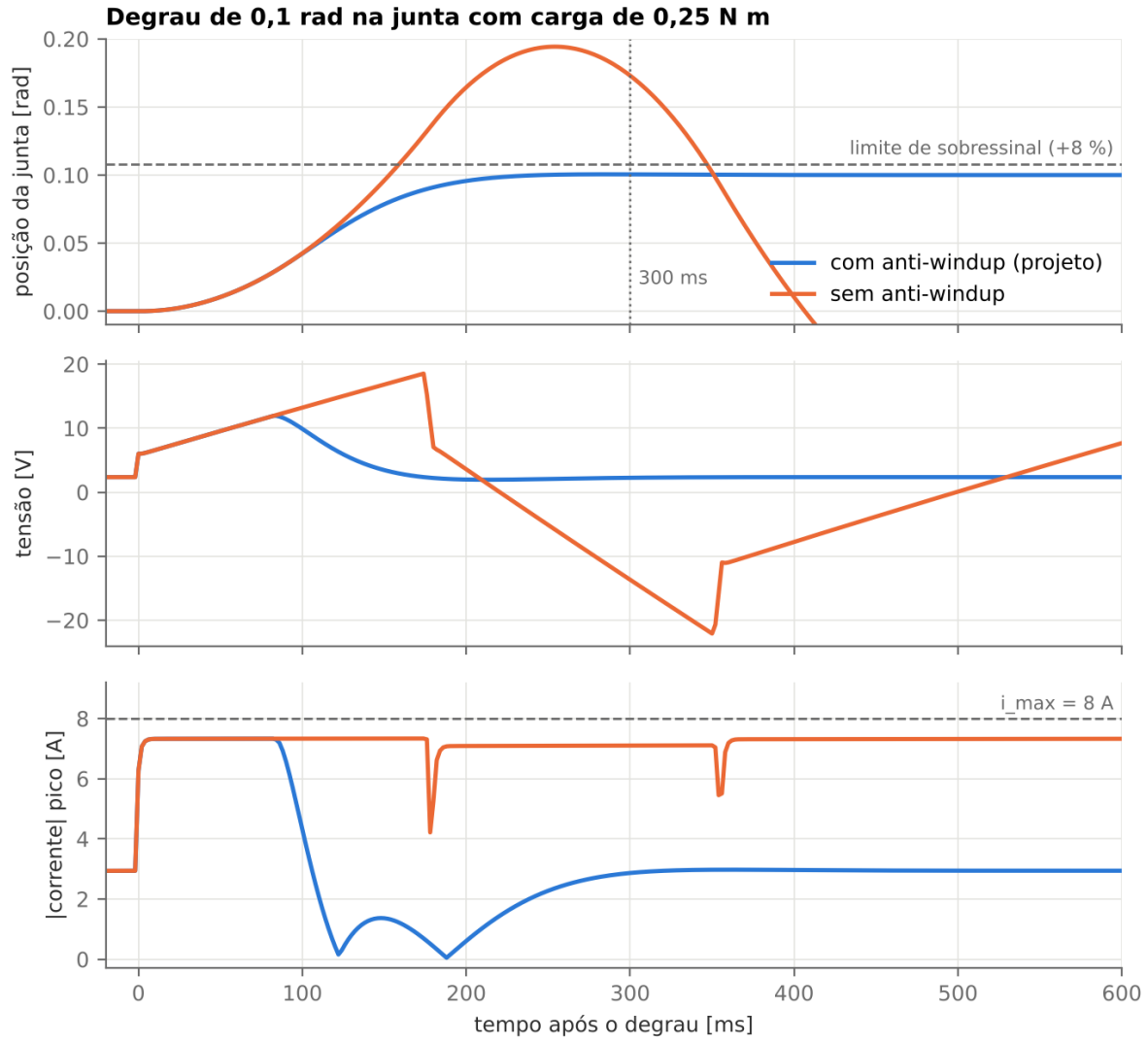


Figura 2 — Degrau de 0,1 rad: com retrocálculo (azul) a junta acomoda em 218 ms com 0,5 % de sobressinal; sem ele (laranja) o integrador acumula erro durante a saturação de corrente e o pico de posição chega a 94 % acima da referência.

Margens. No laço linearizado, com atraso equivalente de 1,5 T_s (ZOH + cálculo), a frequência de cruzamento é 64,5 rad/s (10,3 Hz) e a margem de fase é 51,8°, com 49 amostras por período de cruzamento. Robustez: com J ±20 % todos os requisitos continuam atendidos (`tests/test_controlador.py::test_robustez_inercia`).

Defeito encontrado e corrigido: a primeira versão saturava com min/max combinados. O teste aleatório do SIL mostrou que, se $|K_e \cdot \hat{\omega}| > V_{max} + R \cdot i_{lim}$ (sobrevelocidade ou pulo do encoder), a janela se invertia e o comando chegava a 4 952 V. Agora o limite do barramento é aplicado por último e o teste com 5 000 entradas absurdas garante |V| ≤ 48 V.

8. Supervisor de segurança e verificação formal

8.1 Modelo do supervisor

Cinco estados, nove transições com prioridade, executado a cada ciclo de 2 ms (`auroraarm/supervisor.py`). Em PARADA e AGUARDA_REARME o driver é desabilitado (STO);

a única saída para um estado com torque é T7, que exige rearme. Na RETOMADA o braço volta à trajetória em velocidade colaborativa, partindo da posição medida (sem salto).

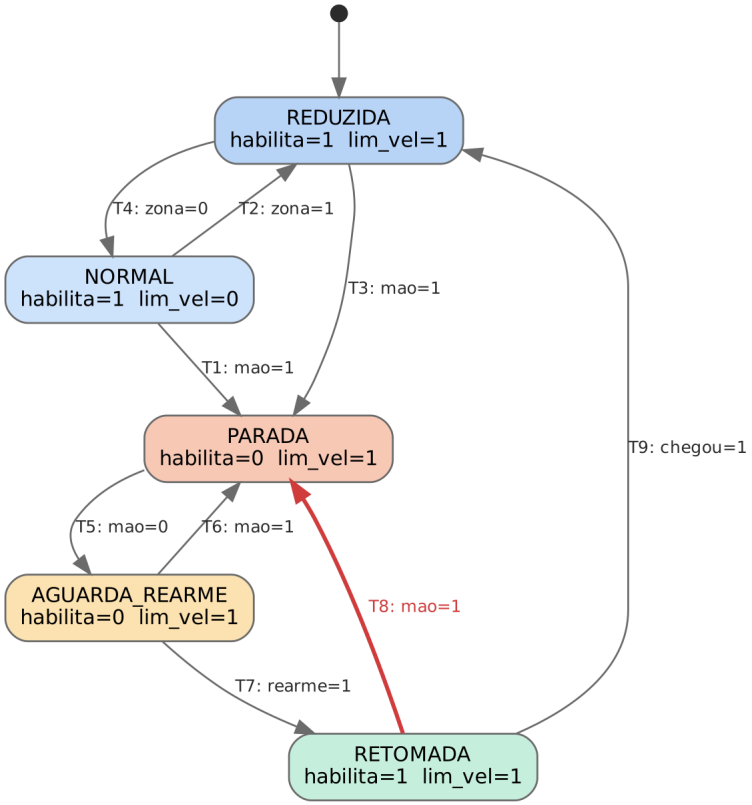


Figura 3 — Supervisor v1. Em vermelho, T8: a transição que faltava na versão preliminar v0 e cuja ausência o model checker encontrou.

8.2 Modelo verificado e premissas

Componente	Comportamento modelado (todas as escolhas são exploradas)
Ambiente	a mão entra/sai do campo de 500 mm a qualquer ms; operador entra/sai da zona; rearme a qualquer ciclo
Scanner	detecção com atraso não determinístico em [0, 20] ms; saída acionada por ≥ 6 ms (A4)
RTOS	tarefa de 2 ms; pode perder 1 ciclo (nunca 2 seguidos); saída vale 1 ms após a ativação (≥ 200 µs)
Driver	habilita = 0 → torque nulo em 8 ms (≥ $5\tau_e$; a simulação dá 6,91 ms até 1 %)
Retomada	termina em até 60 ciclos = 120 ms se não for interrompida (A2)
Premissa A1	a mão permanece no campo até ser detectada (intrusões mais curtas que o tempo de resposta do scanner não são garantidas pelo fabricante)

8.3 Resultado da verificação exaustiva (v1)

Propriedade	Resultado	Detalhe
P1 — REQ-SAFE-AA-001	VERDADEIRA	pior latência entrada da mão → torque nulo: 31 ms (≤ 100)
P2 — REQ-SAFE-AA-002	VERDADEIRA	nenhuma transição religa o torque sem rearme
P3 — REQ-SAFE-AA-003	VERDADEIRA	nenhum ciclo alcançável com obrigação de retomada pendente
P4 — REQ-SAFE-AA-004	VERDADEIRA	pior resposta lógica à zona: 4 ms (≤ 6)
Espaço explorado	32.011 estados	349.694 transições; os 5 estados e as 9 transições do supervisor alcançados

8.4 Contraexemplos obtidos durante o desenvolvimento

C1 — espúrio, de formalização. A primeira versão do monitor de P1 mantinha a obrigação do prazo mesmo depois que a mão saía e o operador rearmava. O traço mostrava o braço operando normalmente com ninguém no campo e acusava violação aos 101 ms. A correção foi no *monitor*: o requisito protege a pessoa enquanto ela está no campo. Lição: um contraexemplo deve ser *lido* antes de corrigir o código.

C2 — real, supervisor v0. A versão preliminar não tinha T8 (premissa implícita: "se o operador acabou de rearmar, a mão está longe"). O model checker devolveu o traço mais curto (109 ms, arquivo `resultados/contraexemplo_v0.txt`):

```
t=  1 ms  mão entra; scanner detecta
t=  4 ms  supervisor (após perder um ciclo): T3 -> PARADA; torque começa a cair
t=  7 ms  mão sai; t=8 ms: T5 -> AGUARDA_REARME
t=  8 ms  MÃO VOLTA, mas o scanner ainda não detectou (atraso de até 20 ms)
t= 10 ms  operador aperta o rearme; sensor = 0  ->  T7 -> RETOMADA, torque religado
t= 11 ms  scanner detecta a mão... v0 não tem transição RETOMADA --mao=1--> PARADA
t=109 ms  mão há 101 ms no campo com torque ligado  <<< VIOLAÇÃO de REQ-SAFE-AA-001
```

É exatamente uma "condição de borda entre os estados de movimento e de parada de segurança", como a do protótipo anterior, e depende de uma combinação de três fatores (atraso do scanner, rearme durante esse atraso e mão que volta) que dificilmente apareceria em teste manual. Com T8 o supervisor v1 volta a PARADA no ciclo seguinte à detecção. Outro aprendizado: com a duração da retomada modelada como 5 ciclos o defeito *não* aparecia, porque a retomada terminava sozinha antes do prazo — a abstração precisa ser mais longa que o prazo verificado (K_{RET} = 120 ms).

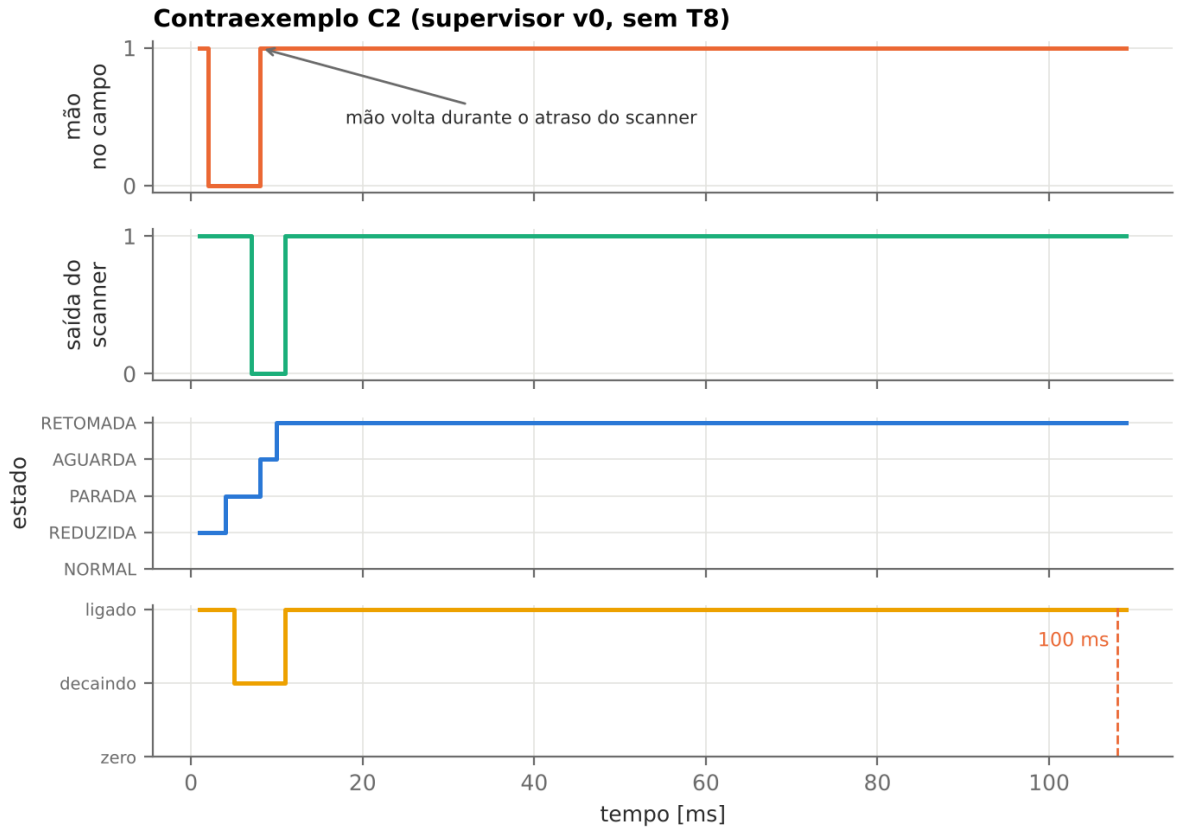


Figura 4 — Contraexemplo C2 na versão v0: a mão volta durante o atraso do scanner, o rearme religa o torque e a detecção posterior é ignorada até estourar o prazo.

8.5 Prazo de 100 ms sob os atrasos informados (memorial, item d)

Além do model checking. a cadeia de latência foi varrida analiticamente em 64.320 combinações (atraso do sensor de 0 a 20 ms em passos de 0,1 ms; instante da detecção dentro do período; 0 ou 1 ciclo perdido; execução de 0 ou 200 µs; corte do torque pela

simulação ou por $5\tau_e$). **Todas** cumprem o prazo; o pior caso é 31,96 ms, com margem de 68,0 ms. O valor inteiro do model checker (31 ms) é coerente com ele (relógios de 1 ms). A violação exigiria 36 ciclos perdidos seguidos ou um scanner com atraso acima de 88,0 ms.

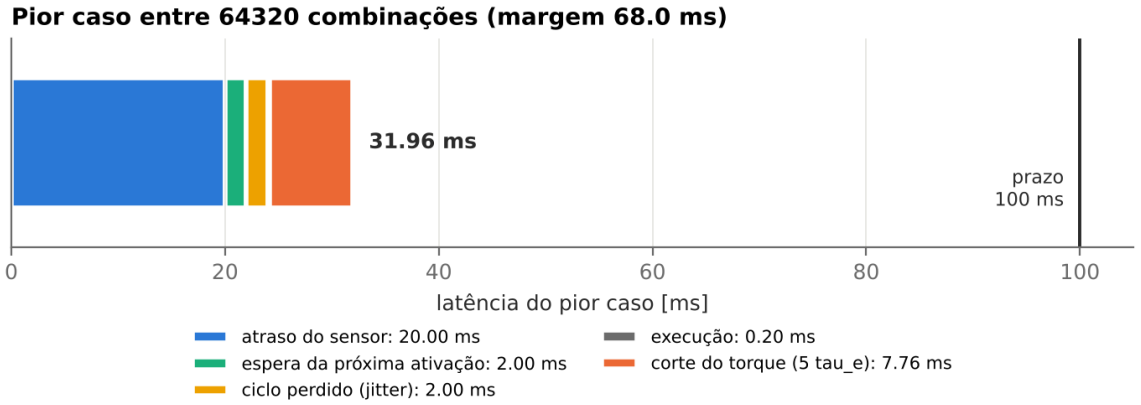


Figura 5 — Decomposição do pior caso de REQ-SAFE-AA-001 (memorial, item d).

8.6 REQ-SAFE-AA-004 na física da junta

A parte lógica (P4) garante que lim_vel liga em até 4 ms. A parte física depende de desacelerar: $v = (\omega/N)\cdot r \le 0{,}25$ m/s exige $\omega \le 71{,}4$ rad/s no motor. A referência colaborativa usa 85 % desse valor (60,7 rad/s) e desacelera a 600 rad/s². Partindo do ponto crítico (1,57 m/s), a junta fica abaixo do limite em 666 ms e não volta a ultrapassá-lo; o pico de corrente é 3,74 A. Por isso o campo de *alerta* do scanner (que sinaliza "zona") deve ficar a pelo menos 1,6 m/s × 0,691 s ≈ **1,10 m** da área de colaboração (ISO 13855) — requisito derivado para o leiaute da célula.

Defeito encontrado e corrigido: com limitação só de velocidade, a referência "freava" instantaneamente, o motor ultrapassava a referência e oscilava, com pico de 0,42 m/s. A versão atual limita também a aceleração da referência.

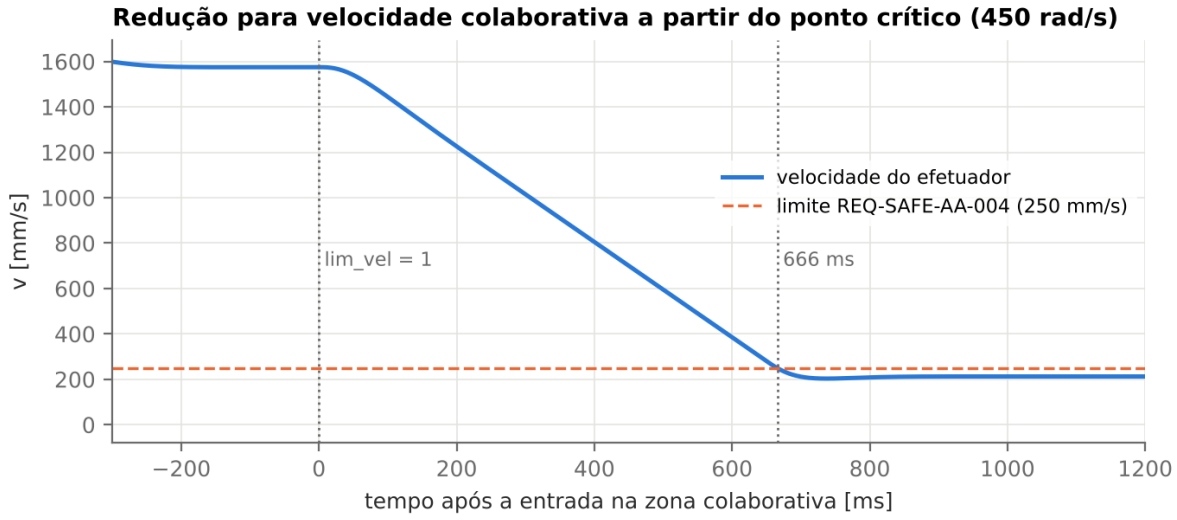


Figura 6 — Entrada na zona colaborativa a 450 rad/s: a velocidade do efetuador fica abaixo de 250 mm/s em 666 ms e estabiliza em 212 mm/s.

9. Testes baseados em modelo e cobertura

Os casos do supervisor são **gerados** a partir da tabela de transições (`auroraarm/testes_modelo.py`) e executados na implementação Python e no C gerado

(`tests/test_supervisor_mbt.py`):

Gerador	Casos	Critério	Python	C gerado
Percurso do grafo (BFS até a origem + disparo)	9	cobertura de transições	9/9 (100 %)	9/9 (100 %)
Pares estado × entrada (5 × 16)	80	todas as transições e não transições	80/80	80/80
Passeio aleatório com invariantes I1–I3	20 000 ciclos	propriedades REQ-SAFE-001/002/004	0 violações	0 violações
Cobertura de estados	—	5 estados	100 %	100 %

Os testes testam a si mesmos. Rodando os casos gerados contra o supervisor v0, o caso MBT-T8 falha (e 797 violações do invariante I1 aparecem no passeio aleatório). Dois mutantes verificam o model checker: sem T9 ele acusa a violação de vivacidade P3; com T5 levando direto à RETOMADA ele acusa P2 (`tests/test_verificacao.py`). A suíte completa tem **42 testes, todos passando**, em cerca de 5 s.

10. Geração de código C e equivalência SIL

Trecho do código gerado (`gen/auroraarm_junta.c`) — note a etiqueta de requisito em cada linha e o bloco de rastreabilidade:

```
/*
 * ==================== BLOCO DE RASTREABILIDADE ====================
 * Arquivo GERADO AUTOMATICAMENTE — não editar à mão.
 * Gerador ........: auroraarm/codegen.py v1.0.0
 * Modelo de origem : auroraarm/controlador.py (PASSOS, coeficientes)
 *                    auroraarm/supervisor.py  (TRANSICOES v1)
 * Parâmetros ......: parametros.json  sha256[:16] = 5e0f44118198114b
 * Hash do modelo ..: 1362dc10ae851da9
 * Requisitos ......: REQ-CTRL-AA-001..004, REQ-SAFE-AA-001..004
 * Verificação .....: tests/test_sil.py (equivalência numérica SIL),
 *                    tests/test_supervisor_mbt.py (testes gerados do grafo),
 *                    auroraarm/verificacao.py (model checking P1..P4)
 * Alvo ............: ARM Cortex-M4F, float IEEE-754 binário32,
 *                    laço de posição a 500 Hz (Ts = 2 ms),
 *                    compilar com -ffp-contract=off
 * ==================================================================
...
    const float t4c = (t4b < AA_V_MAX) ? t4b : AA_V_MAX;   /* REQ-CTRL-AA-004 */
    const float u_sat = (t4c > AA_V_MIN) ? t4c : AA_V_MIN;   /* REQ-CTRL-AA-004 */
    const float u_out = (habilita > 0.5f) ? u_sat : AA_ZERO;   /* REQ-SAFE-AA-001/002 */
```

Equivalência numérica (memorial. item e). Sequência de 3.801 amostras: 801 de um ensaio em malha fechada (degraus, parada de segurança, modo colaborativo) e 3 000 aleatórias em toda a faixa física. Contra o modelo executado em binário32, o erro máximo é **zero**: as saídas são idênticas bit a bit. Contra o modelo em binário64, o erro máximo é 3.61e-04 V, ou seja, 63 ε relativos à escala de 48 V, abaixo da cota teórica de 6.21e-04 V (quantização da posição em binário32 amplificada pelos ganhos derivativo e proporcional) e 65 vezes menor que 1 LSB de um PWM de 12 bits. Em malha fechada com o C, o tempo de acomodação é idêntico ao do modelo (218 ms).

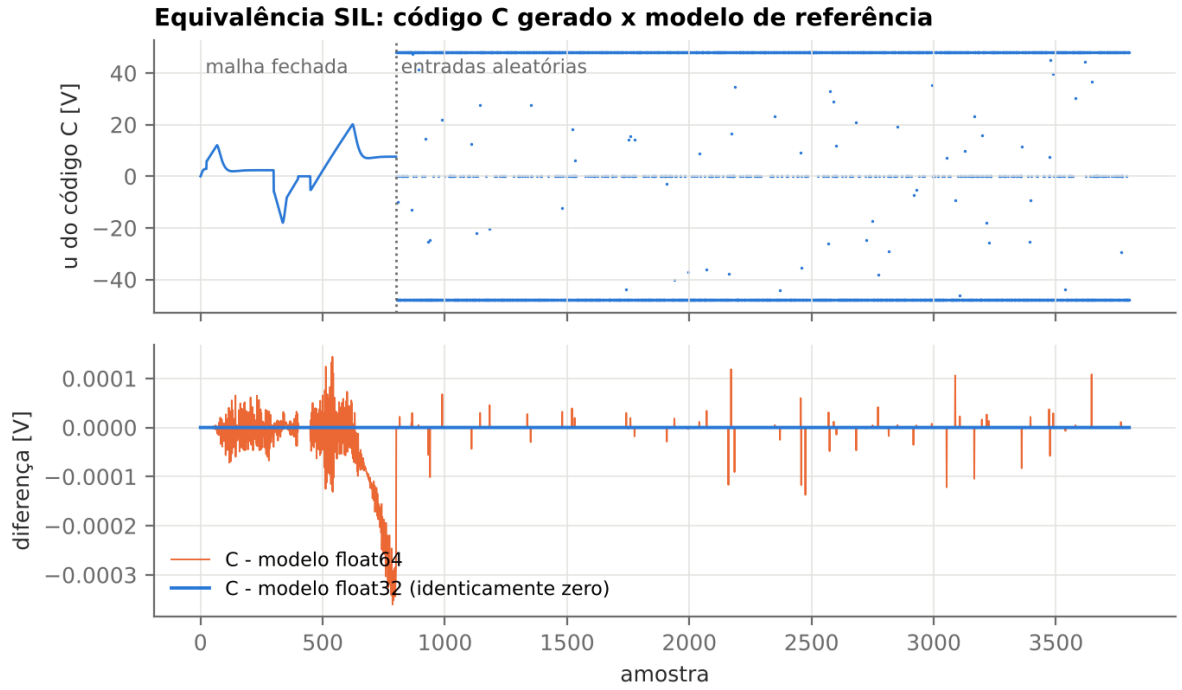


Figura 7 — SIL: diferença entre o código C e o modelo. Em binário32 é identicamente zero; em relação ao binário64 fica na ordem de 10^{-4} V, coerente com o arredondamento.

Recursos no alvo. Compilado com arm-none-eabi-gcc (15:13.2.rel1-2) 13.2.1 para Cortex-M4F (-mfpu=fpv4-sp-d16, -mfloat-abi=hard): **692 bytes** de código (0,26 % dos 256 KB). O controlador e o supervisor somam 219 instruções em linha reta, sem laços nem divisões; assumindo 3 ciclos por instrução (flash com estados de espera), a cota é 657 ciclos = **3,91 µs a 168 MHz**, 2,0 % do orçamento de 200 µs. No host x86-64 o ciclo mede 17,2 ns.

11. Matriz de rastreabilidade

Gerada automaticamente por `scripts/rastreabilidade.py`: as linhas do C são encontradas pelas etiquetas que o gerador grava, e o script falha se algum teste citado não existir. Versão Markdown em `docs/rastreabilidade.md`.

Requisito	Modelo	Código (gen/auroraarm_junta.c)	Teste	Evidência
REQ-CTRL-AA-001 Erro de posição nulo em regime (degrau) *Desempenho*	controlador.py: PASSOS e, t5, t8, i_new (ação integral)	linhas 66, 75-76, 84, 87-88	tests/test_controlador.py::test_req_ctrl_001_a_004	pior erro 1.2e-04 rad em 9 cenários
REQ-CTRL-AA-002 Sobressinal <= 8 % *Desempenho*	controlador.py: br, e_p, p_term (beta = 0,55), d_new (D na medição)	linhas 67, 69-76, 94	tests/test_controlador.py::test_req_ctrl_001_a_004 ::test_robustez_inercia	pior sobressinal 2.81 %
REQ-CTRL-AA-003 Tempo de acomodação <= 300 ms *Desempenho*	controlador.py: ganhos (scripts/sintonia.py) + anti-windup t6, t7	linhas 74-76, 85-86, 88	tests/test_controlador.py::test_req_ctrl_001_a_004 ::test_anti_windup_e_necessario	pior ts 218 ms

Requisito	Modelo	Código (gen/auroraarm_junta.c)	Teste	Evidência
REQ-CTRL-AA-004 |V| <= 48 V e |i| <= 8 A sempre *Desempenho (restrição)*	controlador.py: fcem, v_hi0, v_lo0, t4..u_sat (janela de corrente + barramento)	linhas 67-68, 77-83, 94	tests/test_controlador.py::test_saturacao_nunca_passa_de_48V_mesmo_com_entrada_absurda	pior |i| 7.33 A; |V| <= 48 V com entradas aleatórias
REQ-SAFE-AA-001 Mão < 500 mm -> torque zero em <= 100 ms *Segurança (com prazo)*	supervisor.py: T1, T3, T6, T8; verificacao.py: P1; controlador.py: u_out	linhas 89, 115, 119, 126, 130, 133	tests/test_verificacao.py::test_v1_satisfaz_todas_as_propriedades ::test_v0_tem_contraexemplo_de_prazo	pior latência 31.96 ms (varredura) / 31 ms (model checking)
REQ-SAFE-AA-002 Após parada, só religa com rearme explícito *Segurança*	supervisor.py: T5, T7 (única saída para estado com torque); verificacao.py: P2	linhas 89-91, 123, 127	tests/test_verificacao.py::test_checador_detecta_religamento_sem_rearme tests/test_supervisor_mbt.py::test_invariantes_em_passeio_aleatorio	P2 verdadeira em 32011 estados
REQ-SAFE-AA-003 Mão removida + rearme -> retorna à trajetória *Vivacidade*	supervisor.py: T7, T9; controlador.py: r_prev/v_prev (retomada sem salto); verificacao.py: P3	linhas 64-65, 92-93, 127, 131	tests/test_verificacao.py::test_v1_satisfaz_todas_as_propriedades ::test_checador_detecta_violacao_de_vivacidade	P3 verdadeira (nenhum ciclo com obrigação pendente)
REQ-SAFE-AA-004 Velocidade do efetuador <= 250 mm/s na zona colaborativa *Segurança*	supervisor.py: T2, T4, saída lim_vel; controlador.py: dr..r (limitador vel./acel.); verificacao.py: P4	linhas 54-65, 116, 120	tests/test_controlador.py::test_modo_colaborativo_limita_velocidade tests/test_supervisor_mbt.py::test_invariantes_em_passeio_aleatorio	regime 212 mm/s; desaceleração 666 ms; campo de alerta 1.10 m

12. Honestidade técnica: o que o pacote sustenta e o que falta

O que este pacote sustenta perante a auditoria da Aurora Robótica:

- o modelo da junta é coerente com a física e com os parâmetros de bancada, e o procedimento de identificação recupera esses parâmetros com erro < 1 % a partir de dados ruidosos;
- o controlador atende REQ-CTRL-AA-001..004 em todos os cenários do envelope declarado, com saturação de tensão e de corrente garantidas por construção;
- a *lógica* do supervisor satisfaz REQ-SAFE-AA-001..004 em **todas** as combinações de atraso do scanner, perda de ciclo e ação do operador do modelo — não em uma amostra delas;
- o código que vai para o microcontrolador é gerado do modelo verificado, é equivalente a ele bit a bit em precisão simples e cabe com folga nos limites de tempo e memória;
- cada requisito está ligado a modelo, linhas de código, testes e números reprodutíveis.

O que ainda não sustenta, e o que exigiria para certificação (IEC 61508 ou, se o setor pedir, ISO 26262/ISO 13849):

- **Qualificação de ferramentas.** O model checker, o gerador de código e o gerador de testes foram escritos neste projeto e não são qualificados (IEC 61508-3, 7.4.4; ISO 26262-8, cláusula 11). Seria preciso qualificá-los, ou usar ferramentas qualificadas (UPPAAL/compilador certificado), ou verificar o código gerado de forma independente a cada geração. O compilador GCC também não é qualificado.

- **Independência da verificação.** Modelo, código e testes foram produzidos pela mesma pessoa. Para SIL 2/PL d a verificação deve ser feita por pessoa ou equipe independente.
- **Fidelidade das premissas.** A1–A4, o atraso de 20 ms e o tempo de STO precisam ser confirmados com os dados do scanner e do driver reais; a identificação foi emulada, não feita em bancada; a flexibilidade da harmônica, o atrito de Coulomb e o aquecimento não foram modelados.
- **HIL e hardware.** Falta rodar o binário no Cortex-M4 real (HIL), medir o WCET (a cota foi estimada pela contagem de instruções) e configurar a FPU (sem flush-to-zero). O canal de segurança deveria ser redundante (dois canais, scanner e STO certificados), o que este pacote não cobre.
- **Escopo.** Apenas a junta do cotovelo e apenas a sua contribuição para a velocidade do efetuador; o braço completo exige a cinemática das seis juntas e a análise de força/pressão de contato da ISO/TS 15066 (limitação de potência e força), não tratada aqui.

Referências

BAIER, Christel; KATOEN, Joost-Pieter. **Principles of model checking**. Cambridge: MIT Press, 2008.

COLIM, Ana; FARIA, Carlos; CUNHA, João; OLIVEIRA, João; SOUSA, Nuno; ROCHA, Luís A. Physical ergonomic improvement and safe design of an assembly workstation through collaborative robotics. **Safety**, Basel, v. 7, n. 1, art. 14, 2021. DOI: 10.3390/safety7010014. Disponível em: https://www.mdpi.com/2313-576X/7/1/14. Acesso em: 28 out. 2026.

HENZINGER, Thomas A.; MANNA, Zohar; PNUELI, Amir. What good are digital clocks? In: INTERNATIONAL COLLOQUIUM ON AUTOMATA, LANGUAGES AND PROGRAMMING (ICALP), 19., 1992, Viena. **Proceedings**. Berlin: Springer, 1992. p. 545-558. (Lecture Notes in Computer Science, v. 623).

INTERNATIONAL ELECTROTECHNICAL COMMISSION. **IEC 61508**: functional safety of electrical/electronic/programmable electronic safety-related systems. Parts 1-7. Geneva: IEC, 2010.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/TS 15066**: robots and robotic devices — collaborative robots. Geneva: ISO, 2016.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO 13855**: safety of machinery — positioning of safeguards with respect to the approach speeds of parts of the human body. Geneva: ISO, 2010.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO 26262**: road vehicles — functional safety. Geneva: ISO, 2018.

MODELICA ASSOCIATION. **Functional Mock-up Interface Specification**, version 3.0. Linköping: Modelica Association, 2022.

SICILIANO, Bruno; SCIAVICCO, Lorenzo; VILLANI, Luigi; ORIOLO, Giuseppe. **Robotics**: modelling, planning and control. London: Springer-Verlag, 2010.

UNIFECAF. **Model Based Design for Cyber-Physical Systems**: material das aulas 1 a 16. São Paulo: UniFECAF, 2026.

ANEXO A — Memorial de cálculo

Cada item apresenta fórmula, substituição, premissas e resultado. Os valores são recalculados por `scripts/memorial.py` (saídas em `resultados/memorial.txt` e `resultados/memorial.json`).

(a) Modelo em espaço de estados e constantes de tempo

Premissas: parâmetros da tabela referidos ao eixo do motor; $K_t = K_e$; atrito viscoso; $x = [i, \omega]^T$, $u = V$, perturbação $d = T_L$, $y = \omega$.

Passo 1 — matrizes.

$$a_{11} = -R/L = -0{,}8/0{,}0012 = -666{,}6667\ s^{-1} \qquad a_{12} = -K_e/L = -0{,}085/0{,}0012 = -70{,}8333\ A/(rad)$$

$$a_{21} = K_t/J = 0{,}085/4{,}2\times10^{-4} = 202{,}3810 \qquad a_{22} = -b/J = -1{,}5\times10^{-4}/4{,}2\times10^{-4} = -0{,}357143\ s^{-1}$$

$$B = [1/L\ ;\ 0] = [833{,}3333\ ;\ 0] \qquad E = [0\ ;\ -1/J] = [0\ ;\ -2.380{,}9524] \qquad C = [0 \quad 1]$$

Passo 2 — autovalores. $\det(\lambda I - A) = \lambda^2 - \mathrm{tr}(A)\cdot\lambda + \det(A) = 0$:

$$\mathrm{tr}(A) = -667{,}0238 \qquad \det(A) = a_{11}a_{22} - a_{12}a_{21} = 238{,}0952 + 14.335{,}3175 = 14.573{,}41$$

$$\lambda = [\mathrm{tr} \pm \sqrt{(\mathrm{tr}^2 - 4\det)}]/2 \quad \rightarrow \quad \lambda_1 = -644{,}409\ rad/s, \quad \lambda_2 = -22{,}615\ rad/s$$

Passo 3 — constantes de tempo.

$$\tau_e = 1/|\lambda_1| = 1{,}5518\ ms \quad (\text{aprox. } L/R = 1{,}500\ ms)$$

$$\tau_m = 1/|\lambda_2| = 44{,}218\ ms \quad (\text{aprox. } RJ/(Rb + K_tK_e) = 45{,}745\ ms)$$

Discussão: $\tau_m/\tau_e = 28{,}5$. Os autovalores são reais e bem separados; as aproximações desacopladas erram cerca de 3 % a 3,5 %, o que confirma que as dinâmicas elétrica e mecânica podem ser tratadas em escalas de tempo distintas (identificação em duas etapas, projeto do PID no modelo reduzido, corte do torque quase instantâneo).

Passo 4 — controlabilidade e observabilidade (n = 2):

$$M_c = [B \quad AB] = [833{,}33 \quad -555.555{,}6\ ;\ 0 \quad 168.650{,}8] \qquad \det M_c = 1.4054e{+}08 \quad \rightarrow \quad \text{posto } 2$$

$$M_o = [C\ ;\ CA] = [0 \quad 1\ ;\ 202{,}3810 \quad -0{,}357143] \qquad \det M_o = -K_t/J = -202{,}38 \quad \rightarrow \quad \text{posto } 2$$

Resultado: posto completo nas duas matrizes — a junta é **controlável** pela tensão e **observável** pela velocidade. Como $\det M_o = -K_t/J$, a observabilidade só se perderia se $K_t = 0$ (sem acoplamento eletromecânico). Código: `auroraarm/planta.py`; teste: `tests/test_planta.py`.

(b) Ponto de operação crítico

Premissas: regime permanente ($di/dt = d\omega/dt = 0$), $\omega = 450$ rad/s, $T_L = 0{,}25$ N m, parâmetros nominais.

Passo 1 — corrente (equação mecânica em regime: $K_t\cdot i = b\cdot\omega + T_L$):

$$i_{ss} = (b\cdot\omega + T_L)/K_t = (1{,}5\times10^{-4}\cdot450 + 0{,}25)/0{,}085 = (0{,}0675 + 0{,}25)/0{,}085 = \mathbf{3{,}7353\ A}$$

Passo 2 — tensão (equação elétrica em regime: $V = R\cdot i + K_e\cdot\omega$):

$$V_{ss} = 0{,}8\cdot3{,}7353 + 0{,}085\cdot450 = 2{,}988 + 38{,}25 = \mathbf{41{,}238\ V}$$

Passo 3 — folgas:

$$\Delta V = V_{max} - V_{ss} = 48 - 41{,}238 = \mathbf{6{,}762\ V}\ (14{,}1\ \%)$$

$$\Delta i = 8 - 3{,}735 = \mathbf{4{,}265\ A}\ (i_{ss} = 46{,}7\ \%\ \text{de}\ i_{max})$$

Discussão: 93 % da tensão é consumida pela f.c.e.m.; a velocidade máxima possível com a carga é 528,3 rad/s, só 17 % acima do ponto de operação. Qualquer correção de erro ou aceleração adicional satura o atuador — logo a saturação é o regime *normal* de transitório e o **anti-windup é obrigatório**: sem ele o degrau da seção 7 passa de 94 % de sobressinal e não

acomoda. Em corrente a folga é maior (4,3 A), mas a aceleração também é limitada pelo driver: por isso o controlador satura numa janela de corrente (7,5 A, margem de 0,5 A) além dos 48 V. A folga pequena também desaconselha ganhos muito altos (eles só trocariam desempenho linear por mais tempo saturado).

(c) Escolha do período de amostragem

Premissa: regra prática de 10 a 20 amostras por constante de tempo dominante de malha aberta, e 20 a 40 amostras por período da frequência de cruzamento de malha fechada.

$$N = \tau_m/T_s = 44{,}22/2 = \mathbf{22{,}1\ amostras}$$

$$\text{faixa recomendada: } \tau_m/20 = 2{,}21\ \text{ms} \le T_s \le \tau_m/10 = 4{,}42\ \text{ms}$$

$$\text{malha fechada: } \omega_c = 64{,}5\ \text{rad/s} \rightarrow 2\pi/(\omega_c T_s) = 49\ \text{amostras por período}$$

Resultado: T_s = 2 ms é **adequado**, ligeiramente conservador (22 amostras, logo acima da faixa). Não é desnecessariamente conservador porque: (i) o atraso de 1,5 T_s (ZOH + cálculo) já custa $\omega_c \cdot 1{,}5T_s = 11{,}1°$ de margem de fase no cruzamento, e dobrar T_s dobraria essa perda; (ii) o supervisor roda no mesmo ciclo e cada período perdido entra diretamente na latência de REQ-SAFE-AA-001; (iii) a CPU gasta só 2,0 % do orçamento. Ao mesmo tempo, $\tau_e/T_s = 0{,}78$: a dinâmica elétrica é rápida demais para este laço, e por isso o limite de corrente é imposto por uma janela algébrica de tensão, e não por um laço de corrente.

(d) Verificação do prazo de segurança de 100 ms

Premissas: atraso do scanner $d \in [0, 20]$ ms; a detecção ocorre em qualquer instante do período, logo a espera até a próxima ativação vale $w \in (0, 2]$ ms; no máximo 1 ciclo perdido (+2 ms); execução $e \le 0{,}2$ ms; após habilita = 0 o torque é considerado nulo quando a corrente cai a 1 % (simulado: 6,91 ms) ou, conservadoramente, em $5\tau_e$ = 7,76 ms.

$$L = d + w + 2 \cdot n_{perdidos} + e + t_{corte}$$

$$L_{max} = 20 + 2 + 2 \cdot 1 + 0{,}2 + 7{,}76 = \mathbf{31{,}96\ ms \le 100\ ms}$$

Varredura exaustiva: 201 valores de d × 40 de w × 2 de n × 2 de e × 2 de t_{corte} = 64.320 combinações; todas cumprem; mínimo 6.96 ms; margem no pior caso 68.04 ms. **Autômato temporizado:** o model checker explora as mesmas fontes de atraso em tempo inteiro e encontra pior latência de 31 ms em 32.011 estados.

Combinação que violaria: nenhuma dentro das premissas. Por sensibilidade, seriam necessários $n \ge (100 - 20 - 2 - 0{,}2 - 7{,}76)/2 + 1 =$ **36 ciclos perdidos seguidos**, ou um scanner com atraso > **88,0 ms**. A lógica, porém, *pode* violar o prazo: o contraexemplo C2 da versão v0 (seção 8.4) estoura 100 ms com atrasos dentro das premissas — o tempo só é cumprido se a máquina de estados reagir à detecção em todos os estados com torque.

(e) Equivalência modelo-código

Premissas: tipo float (IEEE-754 binário32). $\varepsilon = 2^{-23} = 1.1921\text{e-}07$; mesma ordem de operações; sem FMA; sequência de 3.801 amostras (801 de malha fechada e 3 000 aleatórias).

$$e_{max} = \max_k |u_C[k] - u_{modelo}[k]|$$

modelo em binário32: $e_{max} =$ **0.0 V** (as 3.801 saídas são idênticas bit a bit)

modelo em binário64: $e_{max} =$ **3.605e-04 V** = 3.605e-04/(48·ε) = 63,0 ε

cota de arredondamento: $(B_D + K_p) \cdot |y|_{max} \cdot \varepsilon = (76{,}67 + 10{,}1) \cdot 60 \cdot 1.192\text{e-}07 = 6.21\text{e-}04$ V

Discussão e resultado: o erro zero contra o modelo em binário32 mostra que o gerador traduziu o modelo *sem defeito*: qualquer erro de geração (operação trocada, constante errada,

ordem diferente) apareceria como diferença de ao menos 1 ulp. A diferença para o binário64 não indica defeito: ela vem da quantização da posição em binário32 (até 60 rad → passo de $3{,}8\times10^{-6}$ rad), amplificada pelo ganho derivativo $B_D = K_d/(T_f+T_s) = 76{,}7$; por isso é ~63 ε e não 1 ε, e fica abaixo da cota de 6.2e-04 V. Comparada ao que importa fisicamente — 1 LSB de PWM de 12 bits = 0,0234 V — é 65 vezes menor. A equivalência é **aceitável**.